

LAPORAN PRAKTIKUM TESTING MODEL CUSTOM CNN PADA DATASET CIFAR-10

Dosen Pengampu:

Bayu Adhi Nugroho, Ph.D



**UIN SUNAN AMPEL
S U R A B A Y A**

Nama : Afrizal Dwi Ahmad

NIM : 09010624001

SISTEM INFORMASI

SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI SUNAN AMPEL SURABAYA

2026

Spesifikasi perangkat yang digunakan:

- OS: Kubuntu 26.04.1 LTS (Resolute Raccoon) x86_64
- CPU: 12th Gen Intel(R) Core(TM) i5-12450HX (12) @ 4.40 GHz
- GPU 1: NVIDIA GeForce RTX 3050 6GB Laptop GPU [Discrete]
- GPU 2: Intel UHD Graphics @ 1.30 GHz [Integrated]
- Memory: 32 GiB

1. Import Library

Library yang digunakan pada notebook testing serupa dengan notebook training, namun lebih ringkas karena tidak memerlukan library untuk pembangunan model dari awal maupun GridSearchCV. Fungsi utama yang digunakan adalah `load_model` untuk memuat model yang telah disimpan sebelumnya.

```
import os
import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

os.environ['PYTHONHASHSEED'] = str(23092026)
random.seed(23092026)
np.random.seed(23092026)

import tensorflow as tf
tf.random.set_seed(23092026)

from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import load_model

from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import classification_report, confusion_matrix
```

Penjelasan pengaturan random seed:

- `os.environ['PYTHONHASHSEED'] = str(23092026)`: Mengatur seed hashing Python untuk reproduibilitas.
- `random.seed(23092026)`: Mengatur seed random Python standar.
- `np.random.seed(23092026)`: Mengatur seed NumPy.
- `tf.random.set_seed(23092026)`: Mengatur seed TensorFlow.
- Semua seed diset ke 23092026 untuk memastikan reproduibilitas hasil eksperimen.

2. Load Data Test

Pada tahap ini, hanya data test yang dimuat dari dataset CIFAR-10. Data test terdiri dari 10.000 gambar yang belum pernah digunakan selama proses training. Label (`y_test`) di-flatten menjadi array 1 dimensi untuk memudahkan perhitungan metrik.

```
_, (X_test, y_test) = cifar10.load_data()
y_test = y_test.flatten() # Eliminates 1D array scalar warnings

print(f"X_test shape: {X_test.shape}")
print(f"y_test shape: {y_test.shape}")

X_test shape: (10000, 32, 32, 3)
y_test shape: (10000,)
```

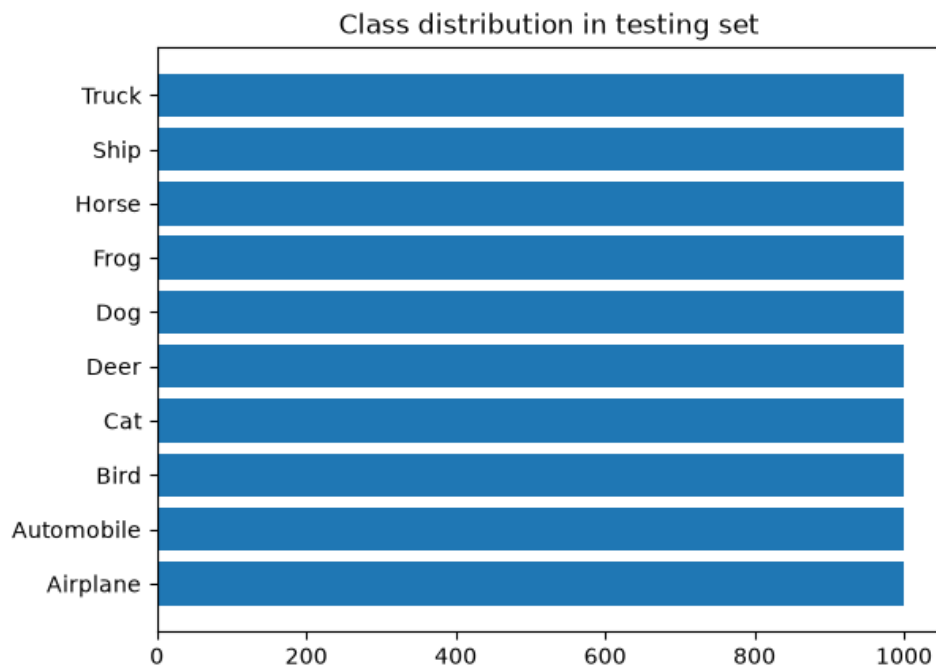
Output menunjukkan bahwa terdapat 10.000 gambar test dengan dimensi 32×32×3 (RGB) dan 10.000 label yang telah di-flatten.

3. Visualisasi Distribusi Kelas

Distribusi kelas pada data test divisualisasikan untuk memastikan data test juga bersifat balanced.

```
classes_name = ['Airplane', 'Automobile', 'Bird', 'Cat', 'Deer', 'Dog', 'Frog', 'Horse', 'Ship', 'Truck']

classes, counts = np.unique(y_test, return_counts=True)
plt.barh(classes_name, counts)
plt.title('Class distribution in testing set')
```



Gambar 1. Distribusi kelas pada data testing CIFAR-10

Dari visualisasi terlihat bahwa setiap kelas memiliki jumlah sampel yang sama, yaitu masing-masing 1.000 gambar. Data test bersifat balanced, sehingga metrik akurasi dapat diandalkan sebagai ukuran performa model.

4. Data Preprocessing

Preprocessing pada data test dilakukan dengan cara yang sama seperti data training: normalisasi piksel ke rentang [0, 1] dan konversi label menjadi one-hot encoding.

```
# Scale the data
X_test = X_test / 255.0

# Transform target variable into one-hotencoding
y_cat_test = to_categorical(y_test, 10)

print(f"X_test shape: {X_test.shape}")
print(f"y_cat_test shape: {y_cat_test.shape}")

X_test shape: (10000, 32, 32, 3)
y_cat_test shape: (10000, 10)
```

Penjelasan preprocessing:

- Normalisasi: Setiap nilai piksel dibagi 255.0 agar konsisten dengan preprocessing pada data training.
- One-Hot Encoding: Label dikonversi ke format vektor biner 10 dimensi menggunakan `to_categorical`, sesuai dengan output layer model yang menggunakan softmax 10 kelas.

5. Load Model yang Telah Dilatih

Model Custom CNN yang telah dilatih dan disimpan dalam format HDF5 (.h5) pada notebook training dimuat kembali menggunakan fungsi `load_model`. Ini memungkinkan evaluasi model tanpa perlu melakukan training ulang.

```
model = load_model('custom_cnn_10_epochs.h5')
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BN)	(None, 32, 32, 32)	128
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
batch_normalization_1 (BN)	(None, 32, 32, 32)	128
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0

dropout (Dropout)	(None, 16, 16, 32)	0	
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496	
batch_normalization_2 (BN)	(None, 16, 16, 64)	256	
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928	
batch_normalization_3 (BN)	(None, 16, 16, 64)	256	
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0	
dropout_1 (Dropout)	(None, 8, 8, 64)	0	
conv2d_4 (Conv2D)	(None, 8, 8, 128)	73,856	
batch_normalization_4 (BN)	(None, 8, 8, 128)	512	
conv2d_5 (Conv2D)	(None, 8, 8, 128)	147,584	
batch_normalization_5 (BN)	(None, 8, 8, 128)	512	
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0	
dropout_2 (Dropout)	(None, 4, 4, 128)	0	
flatten (Flatten)	(None, 2048)	0	
dense (Dense)	(None, 128)	262,272	
dropout_3 (Dropout)	(None, 128)	0	
dense_1 (Dense)	(None, 10)	1,290	

Total params: 552,364 (2.11 MB)

Trainable params: 551,466 (2.10 MB)

Non-trainable params: 896 (3.50 KB)

Optimizer params: 2 (12.00 B)

Model berhasil dimuat dengan arsitektur yang sama seperti yang dibuat pada notebook training. Total parameter: 552.364, dengan tambahan 2 parameter optimizer (Adam) yang tersimpan.

6. Evaluasi Model pada Data Test

Model dievaluasi menggunakan 10.000 gambar test yang belum pernah dilihat selama training. Evaluasi meliputi perhitungan metrik, confusion matrix, dan classification report.

```

labels = ['airplane', 'automobile', 'bird', 'cat', 'deer',
          'dog', 'frog', 'horse', 'ship', 'truck']

# 1. Evaluate metrics on unseen test set
evaluation = model.evaluate(X_test, y_cat_test, verbose=1)
print(f'Test Accuracy : {evaluation[1] * 100:.2f}%')

# 2. Single-pass inference across the complete test set
predictions = model.predict(X_test, batch_size=64, verbose=1)
y_pred = np.argmax(predictions, axis=1)

# 3. Confusion Matrix Display
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)

fig, ax = plt.subplots(figsize=(9, 9))
disp.plot(xticks_rotation='vertical', ax=ax, cmap='summer')
plt.title('Confusion Matrix on Unseen Test Set', fontsize=12)
plt.show()

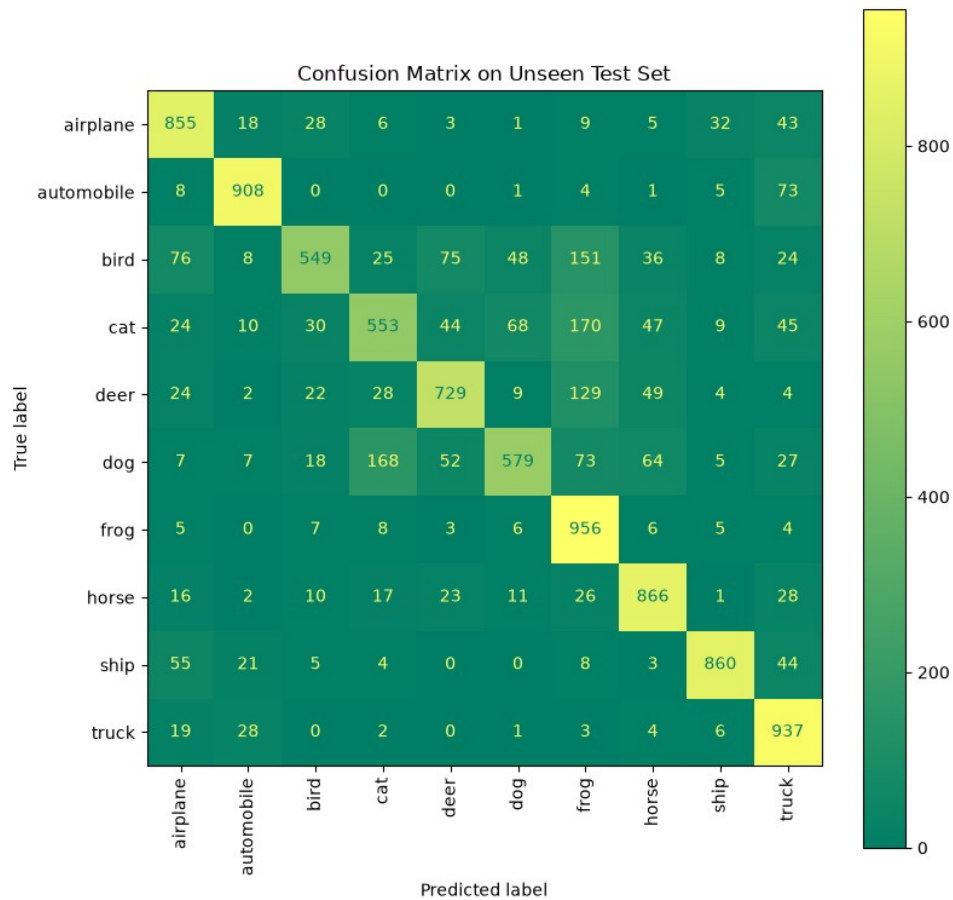
```

313/313 ————— 6s 10ms/step - accuracy: 0.7792 - loss: 0.6642 - precision: 0.8452 - recall: 0.7282

Test Accuracy : 77.92%

157/157 ————— 3s 8ms/step

Model mencapai Test Accuracy sebesar 77.92% pada data test yang belum pernah dilihat. Performa ini konsisten dengan Validation Accuracy (78.19%) yang diperoleh selama training, menunjukkan bahwa model memiliki generalisasi yang baik dan tidak overfitting. Akurasi 77.92% telah melebihi target minimum >75%.



Gambar 2. Confusion Matrix pada data test

Dari confusion matrix terlihat bahwa:

- Kelas "automobile" dan "ship" memiliki performa prediksi terbaik dengan nilai diagonal yang tinggi.
- Kelas "cat" dan "dog" sering tertukar satu sama lain, yang wajar mengingat keduanya memiliki kemiripan visual.
- Kelas "bird" juga mengalami kesulitan klasifikasi, terkadang terklasifikasi sebagai kelas lain.

Classification Report

Classification report memberikan detail metrik precision, recall, dan F1-score untuk setiap kelas:

```
print(classification_report(y_test, y_pred, target_names=labels))
```

	precision	recall	f1-score	support
airplane	0.79	0.85	0.82	1000
automobile	0.90	0.91	0.91	1000
bird	0.82	0.55	0.66	1000
cat	0.68	0.55	0.61	1000
deer	0.78	0.73	0.76	1000
dog	0.80	0.58	0.67	1000
frog	0.63	0.96	0.76	1000
horse	0.80	0.87	0.83	1000
ship	0.92	0.86	0.89	1000
truck	0.76	0.94	0.84	1000
accuracy			0.78	10000
macro avg	0.79	0.78	0.77	10000
weighted avg	0.79	0.78	0.77	10000

Analisis per kelas:

- Automobile: Performa terbaik dengan F1-score 0.91. Precision 0.90 dan Recall 0.91 menunjukkan model sangat baik dalam mengenali kendaraan.
- Ship: F1-score 0.89 dengan Precision tertinggi (0.92), menunjukkan sangat sedikit false positive.
- Truck: F1-score 0.84 dengan Recall tertinggi (0.94), menunjukkan model jarang melewatkan gambar truck.
- Horse: F1-score 0.83, performa konsisten antara precision dan recall.
- Airplane: F1-score 0.82, performa baik dan seimbang.
- Frog: F1-score 0.76. Recall sangat tinggi (0.96) namun Precision rendah (0.63), menunjukkan banyak gambar lain yang salah diprediksi sebagai frog.
- Deer: F1-score 0.76, performa moderat.
- Dog: F1-score 0.67. Recall rendah (0.58) menunjukkan banyak gambar anjing yang tidak terdeteksi.
- Bird: F1-score 0.66. Recall hanya 0.55, menunjukkan hampir setengah gambar burung tidak terklasifikasi dengan benar.
- Cat: Performa terendah dengan F1-score 0.61. Precision (0.68) dan Recall (0.55) rendah, kemungkinan karena kemiripan visual dengan kelas dog.

7. Prediksi pada Sampel Gambar Individu

Untuk demonstrasi, model diuji pada satu gambar test (index ke-100) untuk melihat apakah prediksi model sesuai dengan ground truth.

```
sample_idx = 100
my_image = X_test[sample_idx]

plt.figure(figsize=(3, 3))
plt.imshow(my_image)
plt.title(f"Ground Truth: {labels[y_test[sample_idx]]}")
plt.axis('off')
plt.show()

# Directly reuse the precomputed prediction array
pred_class_idx = np.argmax(predictions[sample_idx])
print(f"Image {sample_idx} Ground Truth Index: {y_test[sample_idx]} ({labels[y_test[sample_idx]]}")
print(f"Model Prediction Index      : {pred_class_idx} ({labels[pred_class_idx]}")
```



Gambar 3. Sampel gambar test ke-100 (Ground Truth: deer)

```
Image 100 Ground Truth Index: 4 (deer)
Model Prediction Index      : 4 (deer)
```

Model berhasil memprediksi gambar ke-100 dengan benar sebagai "deer" (rusa), sesuai dengan ground truth-nya.

8. Visualisasi Prediksi Batch (15 Sampel)

Berikut adalah visualisasi prediksi model pada 15 sampel gambar test pertama. Setiap gambar ditampilkan bersama dengan label prediksi, tingkat kepercayaan (confidence), dan grafik distribusi probabilitas untuk semua kelas. Label berwarna biru menunjukkan prediksi benar, sedangkan warna merah menunjukkan prediksi salah.

```

def plot_image(i, predictions_array, true_label, img):
    predictions_array, true_label, img = predictions_array, true_label[i], img[i]
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(img)
    predicted_label = np.argmax(predictions_array)
    if predicted_label == true_label:
        color = 'blue'
    else:
        color = 'red'
    plt.xlabel(f'{labels[int(predicted_label)]} {100*np.max(predictions_array):2.0f}% ({labels[int(true_label)]})',
               color=color)

def plot_value_array(i, predictions_array, true_label):
    predictions_array, true_label = predictions_array, int(true_label[i])
    plt.grid(False)
    plt.xticks(range(10))
    plt.yticks([])
    thisplot = plt.bar(range(10), predictions_array, color="#777777")
    plt.ylim([0, 1])
    predicted_label = np.argmax(predictions_array)
    thisplot[predicted_label].set_color('red')
    thisplot[true_label].set_color('blue')

# Render predictions for 15 test samples
num_rows = 5
num_cols = 3
num_images = num_rows * num_cols

plt.figure(figsize=(2 * 2 * num_cols, 2 * num_rows))
for i in range(num_images):
    plt.subplot(num_rows, 2 * num_cols, 2 * i + 1)
    plot_image(i, predictions[i], y_test, X_test)
    plt.subplot(num_rows, 2 * num_cols, 2 * i + 2)
    plot_value_array(i, predictions[i], y_test)

plt.tight_layout()
plt.show()

```



Gambar 4. Visualisasi prediksi model pada 15 sampel gambar test

Dari visualisasi prediksi batch terlihat bahwa mayoritas prediksi model bernilai benar (label biru). Grafik bar di sebelah kanan setiap gambar menunjukkan distribusi probabilitas prediksi, dimana bar merah menunjukkan kelas yang diprediksi dan bar biru menunjukkan kelas yang sebenarnya. Ketika keduanya berada pada bar yang sama, prediksi model tepat.

9. Kesimpulan

- Model Custom CNN berhasil dimuat (load) dari file custom_cnn_10_epochs.h5 yang telah disimpan pada proses training.
- Pada data test yang belum pernah dilihat, model mencapai Test Accuracy sebesar 77.92%, melebihi target minimum >75%.
- Performa test (77.92%) konsisten dengan performa validasi (78.19%), menunjukkan model memiliki generalisasi yang baik tanpa overfitting.
- Kelas dengan performa terbaik: automobile (F1=0.91), ship (F1=0.89), truck (F1=0.84).
- Kelas dengan performa terendah: cat (F1=0.61), bird (F1=0.66), dog (F1=0.67) — kemungkinan disebabkan kemiripan visual antar kelas.

- Macro average: Precision 0.79, Recall 0.78, F1-score 0.77.
- Prediksi pada sampel individu dan batch menunjukkan model mampu mengklasifikasikan gambar dengan tingkat kepercayaan yang cukup tinggi.